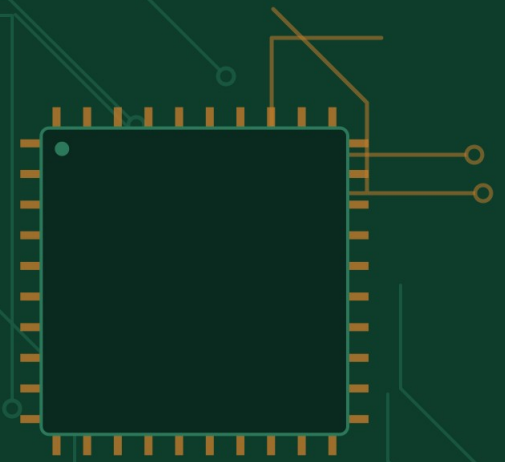
PERTEMUAN 14

Hemat Daya dan Keandalan Sistem

Deep sleep, umur baterai, NVS, dan watchdog



Sistem Tertanam (Embedded Systems) · S1 Teknik Elektro

Dosen Pengampu: Anton Prafanto

Tujuan Pembelajaran

- Membandingkan **mode daya** ESP32 beserta arusnya
- Membuat program **deep sleep** yang bangun oleh timer/tombol
- Menghitung **umur baterai** dari arus rata-rata
- Menyimpan pengaturan di **NVS** (Preferences) dengan aman
- Memakai **watchdog** dan kebiasaan andal lainnya

Peta Materi Hari Ini

01 Mengapa hemat daya dan mode daya ESP32

02 Deep sleep dan sumber bangun

03 Menghitung umur baterai

04 NVS: data yang tidak hilang

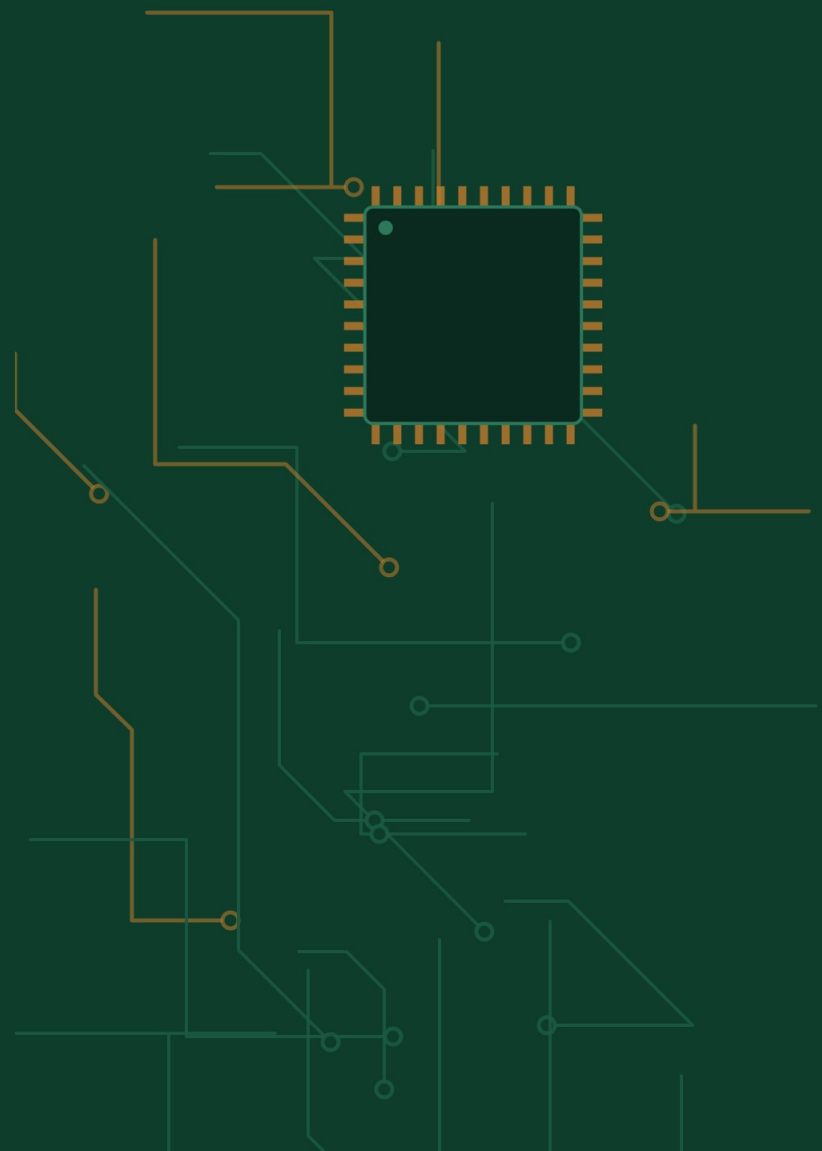
05 Watchdog dan pertahanan berlapis

06 Praktikum 1–4

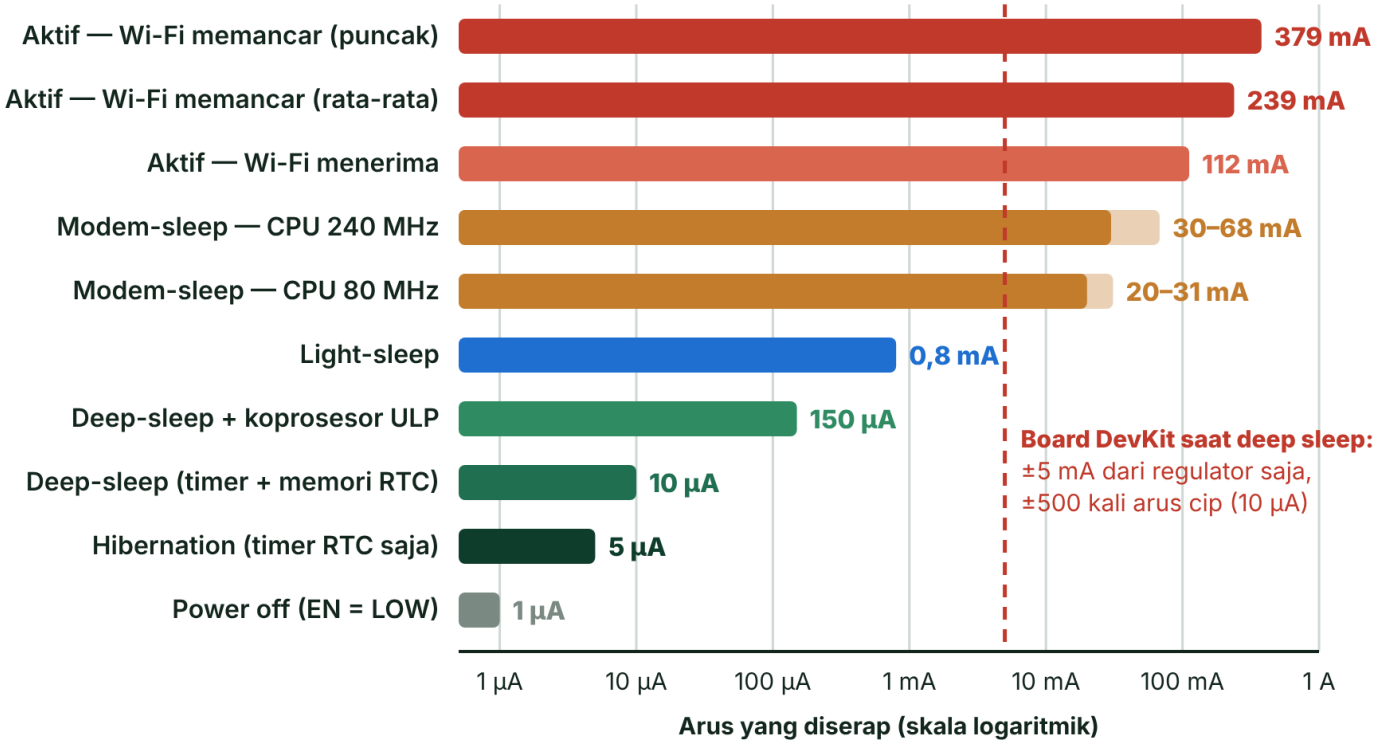
01

Mode Daya dan Deep Sleep

Bekerja sebentar, tidur lama



Tangga Mode Daya ESP32



Sumber: *ESP32 Series Datasheet* v5.3, Tabel 4-2 (arus cip); *ESP32-WROOM-32E Datasheet* (arus Wi-Fi modul); *LM1117 Datasheet*, TI (arus diam regulator keluarga 1117).

Dari Wi-Fi menerima (±112 mA) ke deep sleep (10 µA): arus turun lebih dari 10.000 kali.

Apa yang Tetap Hidup Saat Deep Sleep?

Cip ESP32 saat *deep sleep*

MATI (tanpa daya)

CPU 2 inti
Program berhenti total.

SRAM 520 KB
Isi variabel biasa hilang.

Radio Wi-Fi & Bluetooth
Koneksi terputus.

Periferal digital
UART, I2C, SPI, LEDC/PWM, dan lainnya.

TETAP HIDUP (domain RTC)

Pengendali RTC + timer RTC
Membangunkan cip saat waktunya tiba.

Memori RTC (16 KB)
Tempat variabel `RTC_DATA_ATTR`.

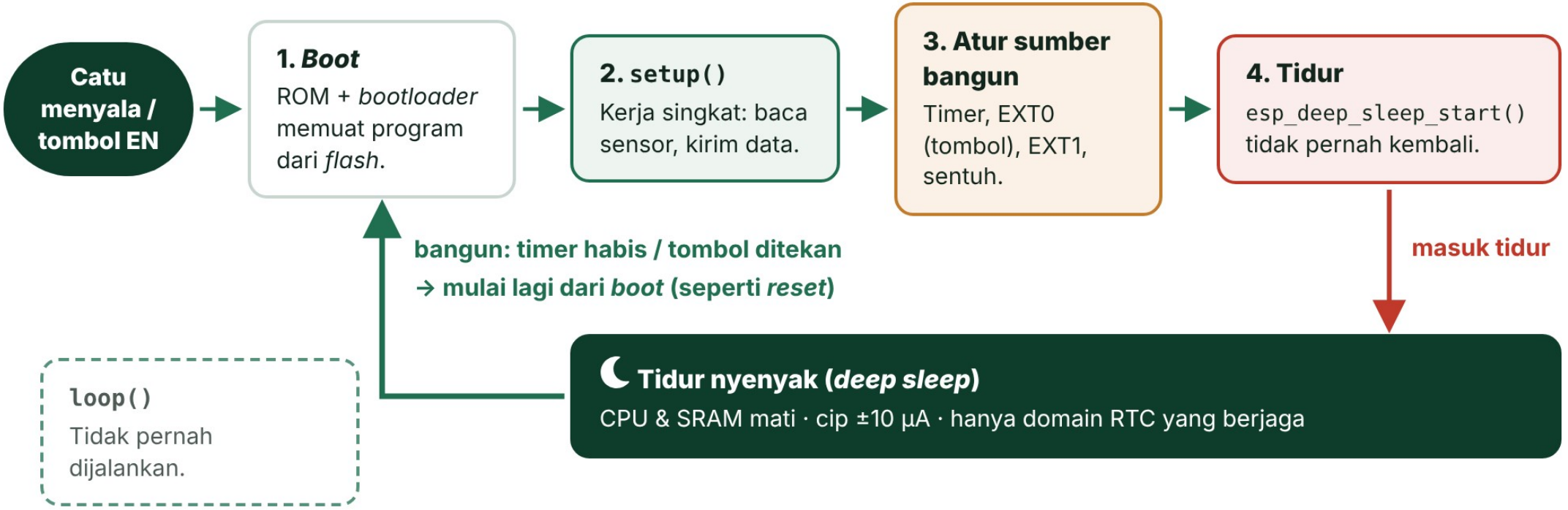
RTC GPIO & sensor sentuh
Sumber bangun EXT0, EXT1, dan sentuh.

Koprosesor ULP (opsional)
Bisa memantau sensor saat tidur (±150 µA).

Arus cip ±10 µA (timer RTC + memori RTC)

Bangun = mulai lagi dari *boot* → `setup()`

Bangun = Mulai Lagi dari Awal



Variabel biasa (SRAM): kembali ke nilai awal tiap bangun `RTC_DATA_ATTR`: nilainya tetap

Praktikum 1: Deep Sleep dengan Timer

kode/01-deep-sleep-timer/src/main.cpp

```cpp
RTC_DATA_ATTR int jumlahBoot = 0;  // memori RTC

void setup() {
  Serial.begin(115200);
  jumlahBoot++;
  Serial.printf("Boot ke-%d\n", jumlahBoot);
  // ... kerja singkat ...
  esp_sleep_enable_timer_wakeup(10ULL * 1000000);
  Serial.flush();
  esp_deep_sleep_start();  // tidak kembali
}

void loop() { }  // tidak pernah jalan
```

- `RTC_DATA_ATTR`: bertahan selama deep sleep
- Satuan timer: **mikrodetik**
- `Serial.flush()` sebelum tidur
- Tombol EN → nilai RTC kembali ke awal

Sumber Bangun dari Deep Sleep

Sumber	Fungsi	Catatan
Timer	esp_sleep_enable_timer_wakeup(us)	Satuan mikrodetik
EXT0	esp_sleep_enable_ext0_wakeup(pin, lvl)	Satu RTC GPIO, pull-up RTC
EXT1	esp_sleep_enable_ext1_wakeup_io(mask, mode)	ANY_HIGH atau ALL_LOW
Sentuh	touchSleepWakeUpEnable(pin, ambang)	10 kanal sentuh
ULP	esp_sleep_enable_ulp_wakeup()	Lanjutan, ±150 µA

Praktikum 2: Bangun dengan Tombol (EXT0)

kode/02-bangun-tombol/src/main.cpp

```
rtc_gpio_deinit(PIN_BANGUN);       // setelah bangun
pinMode(PIN_BANGUN, INPUT_PULLUP);
// ... tunggu tombol dilepas (maks. 3 s) ...

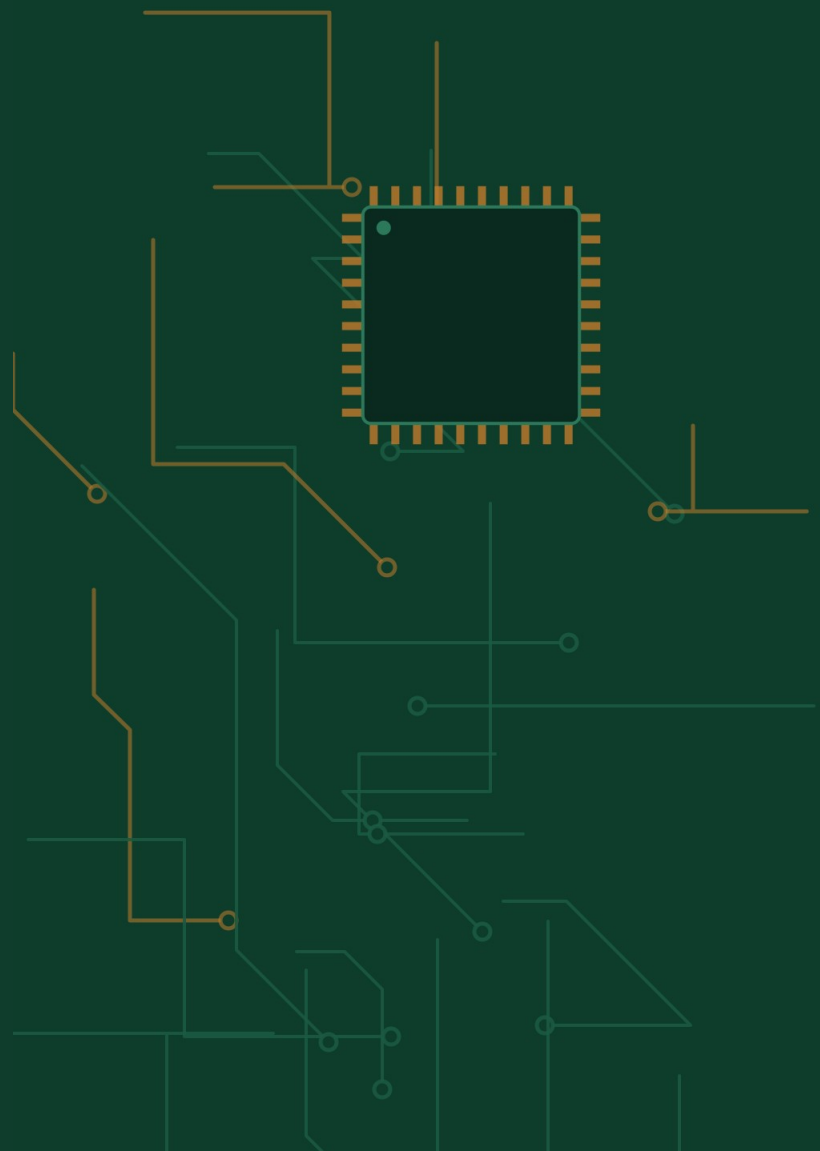
esp_sleep_enable_timer_wakeup(60ULL * 1000000);
rtc_gpio_pullup_en(PIN_BANGUN);    // aktif saat tidur
rtc_gpio_pulldown_dis(PIN_BANGUN);
esp_sleep_enable_ext0_wakeup(PIN_BANGUN, 0);
Serial.flush();
esp_deep_sleep_start();
```

- Hanya **RTC GPIO**; kursus memakai GPIO 33
- Pull-up diatur dengan `rtc_gpio_pullup_en()`
- Paling andal: 10 kΩ eksternal ke 3V3
- Tunggu tombol dilepas, **dengan batas waktu**

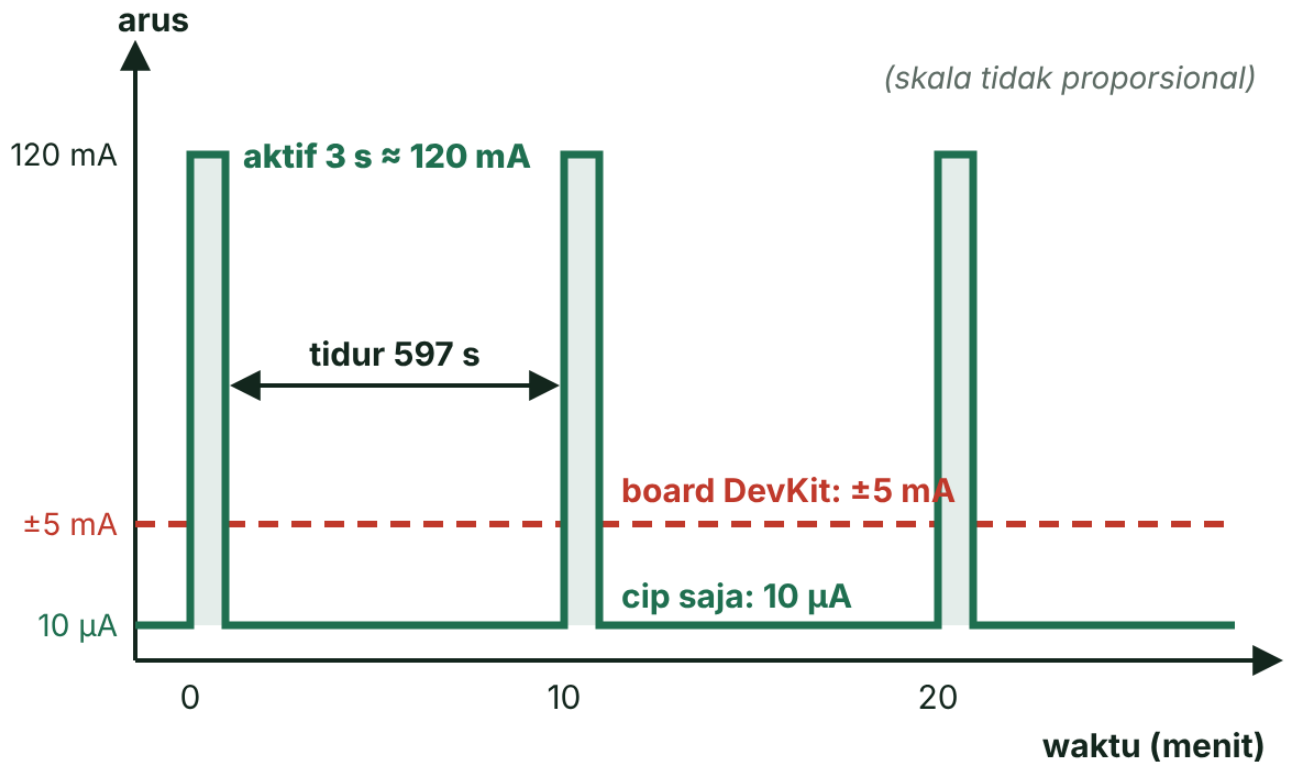
02

Menghitung Umur Baterai

Rata-rata berbobot waktu



Arus Rata-Rata dan Umur Baterai



Contoh: baterai 2.000 mAh, kirim data tiap 10 menit

I_rata = (I_aktif·t_aktif + I_tidur·t_tidur) / T
Umur (jam) = kapasitas (mAh) / I_rata (mA)

Cip ideal (tidur 10 µA):
(120×3 + 0,01×597) / 600 ≈ **0,61 mA**
→ 2.000 / 0,61 ≈ **3.279 jam ≈ 137 hari**

Board DevKit (tidur ±5 mA):
(120×3 + 5×597) / 600 ≈ **5,58 mA**
→ ≈ **359 jam ≈ 15 hari**

Tanpa tidur (120 mA terus): ≈ **17 jam**

Angka aktif 120 mA selama 3 s adalah asumsi contoh; ukur sendiri untuk proyekmu.

Catatan Jujur: Board DevKit



Arus deep sleep **10 µA** milik cip, bukan board.

- Regulator keluarga 1117: arus diam tipikal **5 mA** (LM1117, TI)
- LED daya dan cip USB-UART ikut menyerap arus
- Board tidur ±5 mA → 2.000 mAh hanya ±15 hari
- Ukur sendiri: multimeter seri, mulai rentang mA

Tips Menurunkan Konsumsi



Matikan Wi-Fi

`WiFi.mode(WIFI_OFF)` saat tidak perlu.



Turunkan CPU

`setCpuFrequencyMhz(80)`: 20–31 mA vs 30–68 mA.



Kirim berkelompok

Kumpulkan data di RTC, kirim sekaligus.



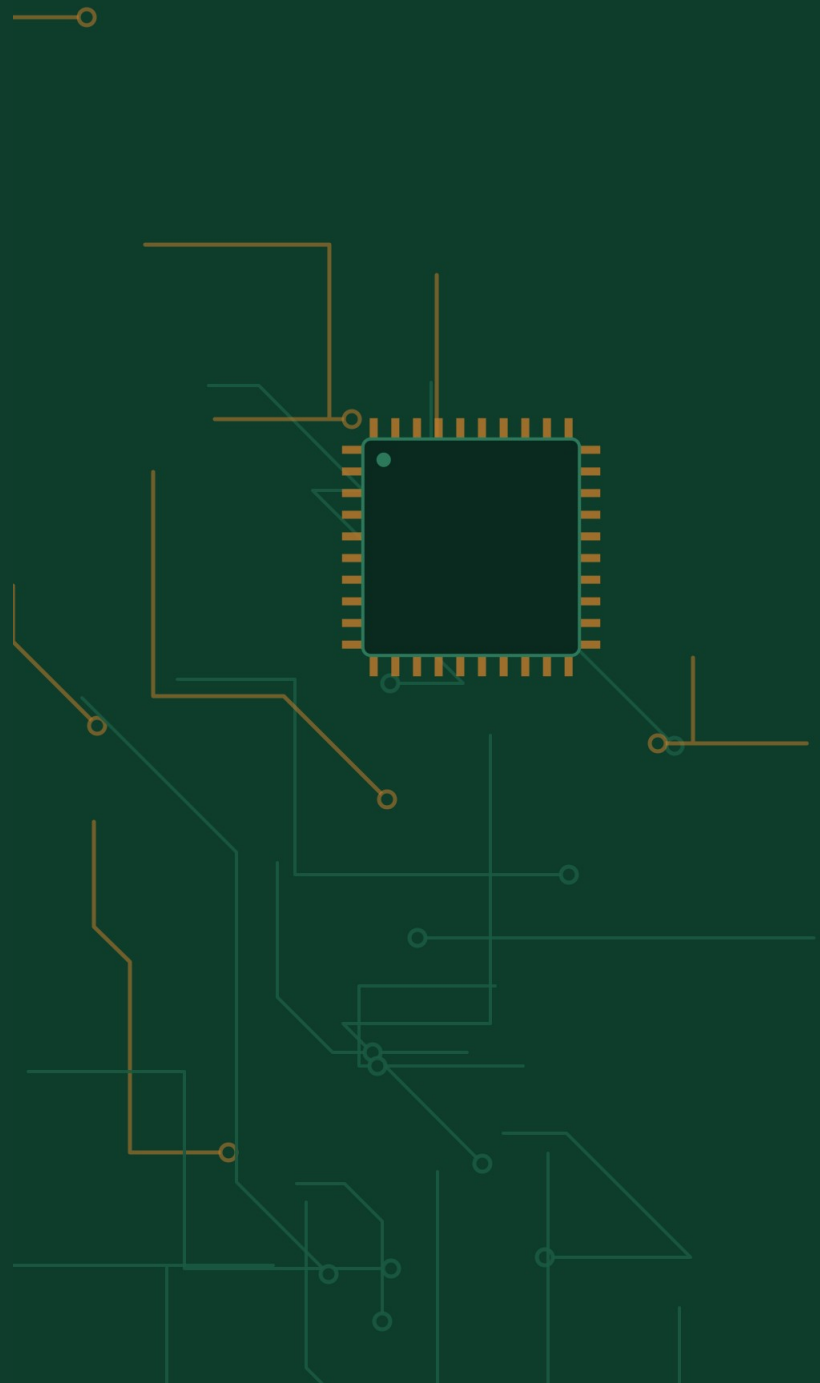
Persingkat bangun

Batas waktu Wi-Fi, hindari `delay()` panjang.

03

Menyimpan Data dan Keandalan

NVS, watchdog, dan pertahanan berlapis



Data Mana yang Bertahan?

	Bangun dari deep sleep	Reset: tombol EN, ESP.restart(), watchdog	Catu dicabut	Unggah program baru
Variabel biasa SRAM 520 KB · cepat, tetapi mudah hilang	✗	✗	✗	✗
`RTC_DATA_ATTR` Memori RTC · cepat, khusus siklus tidur–bangun	✓	✗	✗	✗
NVS / Preferences Flash · lebih lambat, jumlah hapus-tulis terbatas	✓	✓	✓	✓ *

* NVS tetap ada selama *flash* tidak dihapus penuh (*Erase Flash*) dan tabel partisi tidak diubah. Bangun dari deep sleep pun membaca NVS dengan normal.

Praktikum 3: Pengaturan di NVS

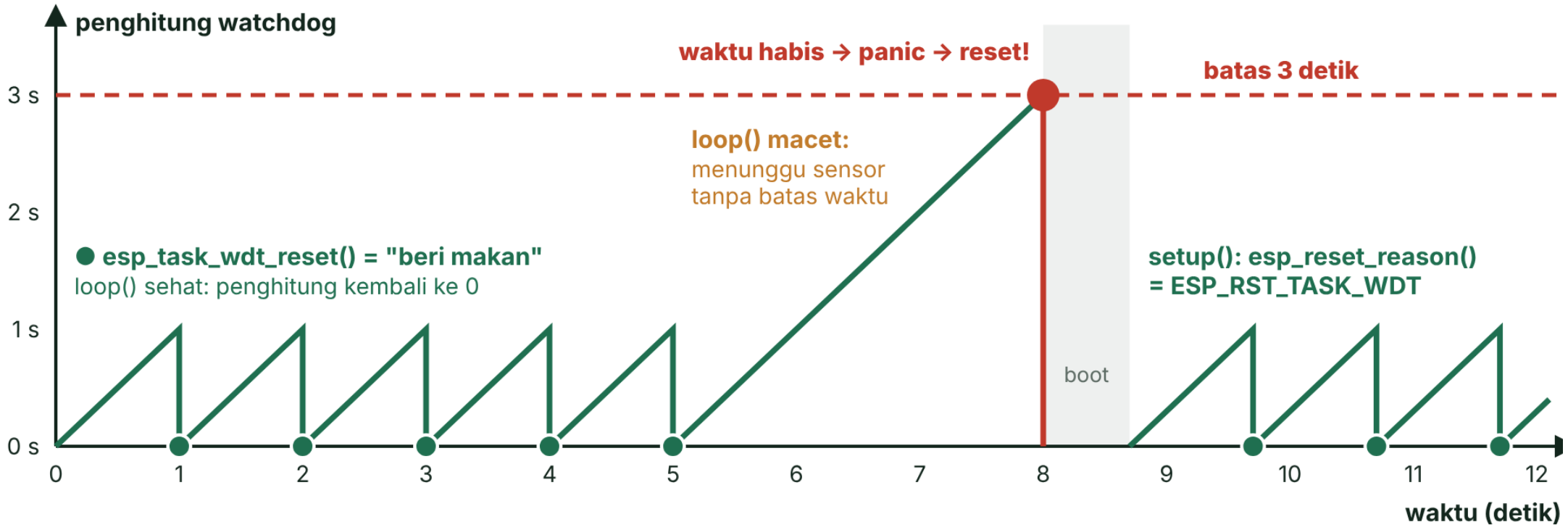
kode/03-simpan-pengaturan-nvs/src/main.cpp

```cpp
#include <Preferences.h>
Preferences pengaturan;

void setup() {
  pengaturan.begin("pengaturan", false);
  uint32_t iv = pengaturan.getUInt("interval", 500);
  // validasi: di luar 100-5000 ms -> pakai 500
  if (iv != 250) {                // tulis bila berubah
    pengaturan.putUInt("interval", 250);
  }
  pengaturan.end();
}
```

- *Namespace* dan *key* maks. **15 karakter**
- `getUInt(key, bawaan)` → nilai bawaan aman
- Tulis **hanya bila berubah**: flash bisa aus
- Partisi NVS bawaan Arduino: 20 KB

Watchdog: Pedal Kewaspadaan Program



Praktikum 4: Task Watchdog

kode/04-watchdog/src/main.cpp

```cpp
// di setup()
esp_task_wdt_config_t k;
k.timeout_ms = 3000;         // batas 3 detik
k.idle_core_mask = 1 << 0;   // idle task inti 0
k.trigger_panic = true;      // habis -> reset
esp_task_wdt_reconfigure(&k);
esp_task_wdt_add(NULL);      // awasi loopTask

// di loop()
esp_task_wdt_reset();        // "beri makan"
```

- Bawaan: TWDT 5 s, hanya *idle task* inti 0
- `loop()` diawasi bila didaftarkan
- Cara singkat: `enableLoopWDT()`
- Beri makan hanya saat program benar-benar maju

Mengetahui Penyebab Reset

Nilai esp_reset_reason()	Arti
ESP_RST_POWERON	Catu baru tersambung (tombol EN juga, di ESP32)
ESP_RST_SW	ESP.restart() dari program
ESP_RST_PANIC	Program crash (exception/panic)
ESP_RST_TASK_WDT	Task Watchdog: program macet
ESP_RST_DEEPSLEEP	Bangun dari deep sleep
ESP_RST_BROWNOUT	Tegangan catu sempat turun

Pertahanan Berlapis

1

Validasi input

Cek rentang nilai dari Serial, jaringan, sensor, dan NVS.

2

Batas waktu

Setiap penantian punya batas, pakai millis().

3

Nilai bawaan aman

Data hilang atau rusak → nilai yang tidak membahayakan.

4

Catat penyebab reset

esp_reset_reason() + NVS; watchdog jaring terakhir.

Praktikum Hari Ini

Praktikum	Inti	Wokwi
1 · Deep sleep timer	RTC_DATA_ATTR, penyebab bangun	Terbatas
2 · Bangun tombol	EXT0 GPIO 33 + pull-up RTC	Terbatas
3 · Pengaturan NVS	Preferences + perintah Serial	Bisa
4 · Watchdog	TWDT 3 s + esp_reset_reason()	Bisa

Kuis Cepat

`RTC_DATA_ATTR int n` bernilai 7, lalu tombol EN ditekan saat ESP32 tidur. Berapa n saat setup() dimulai?

A 7

B 8

C 0

D Acak

Rangkuman

- Bekerja sebentar, tidur lama: deep sleep cip ±10 µA
- Bangun dari deep sleep = mulai lagi dari `setup()`
- Umur baterai ≈ kapasitas / arus rata-rata berbobot waktu
- NVS bertahan saat catu dicabut; tulis hanya bila perlu
- Watchdog + validasi + batas waktu + nilai bawaan aman

Sebelum Pertemuan 15



Tugas

- Kerjakan latihan stasiun cuaca hemat daya
- Hitung umur baterai rancangan proyekmu



Persiapan

- Baca modul Pertemuan 15: Integrasi Sistem
- Tentukan ide proyek akhir



Butuh Bantuan?

- Baca panduan/06-mengatasi-masalah.md
- Tanyakan di forum kelas